2026後期　輪講

小島晴貴（M2）

名古屋大学工学研究科
機械システム工学専攻
モビリティシステム研究グループ

2026/10/23

目次

◆ 自身の研究概要

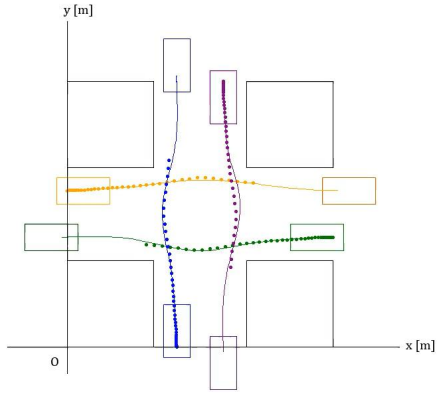
◆ 論文1: Distributed model predictive formation control

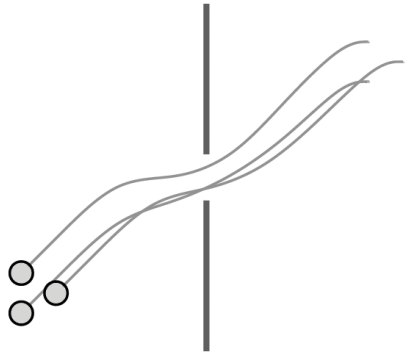
with inter-vehicle collision avoidance

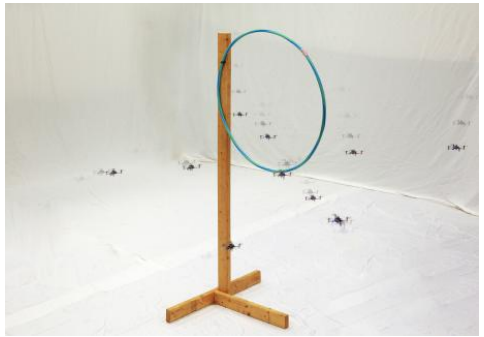
◆ 論文2: Online Trajectory Generation with

Distributed Model Predictive Control

for Multi-Robot Motion Planning







自身の研究概要

自動運転における難しいタスク

狭い/混雑環境下での
障害物や他車両の回避



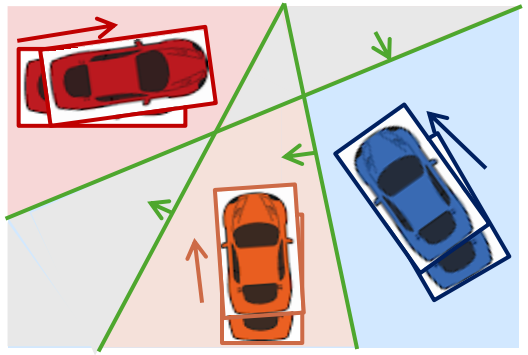
狭い交差点
https://www.kobe-np.co.jp/news/richb/202504/0018900727.shtml



狭い物流倉庫
https://ascii.jp/elem/000/001/842/1842748/

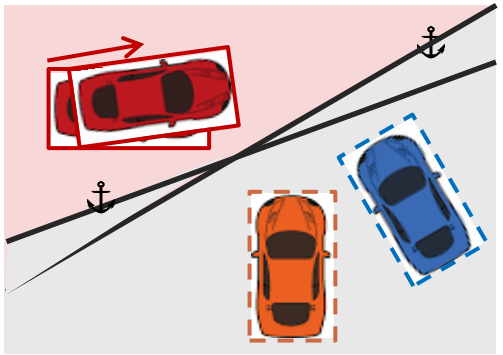
物体間の分離超平面で衝突回避する手法を提案

集中型

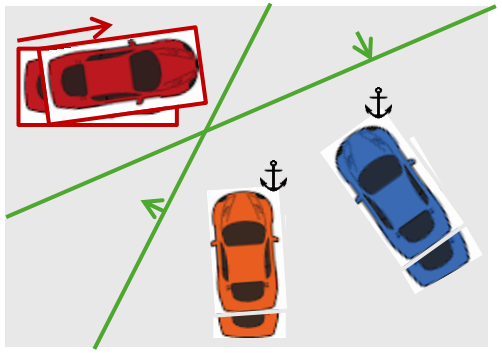


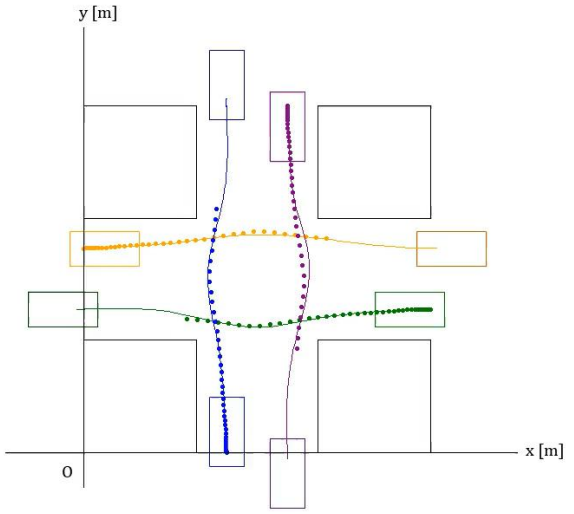
分散型

境界共有



軌道共有

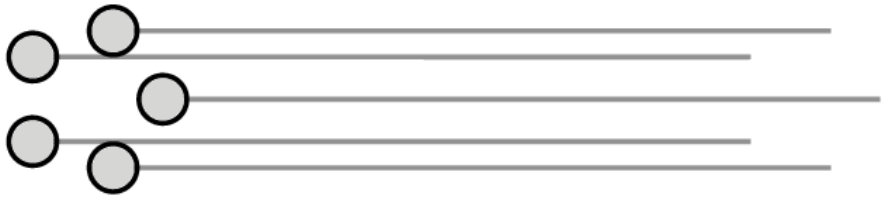




車両間協調・リアルタイム性に課題

論文1：概要

Distributed model predictive formation control with inter-vehicle collision avoidance



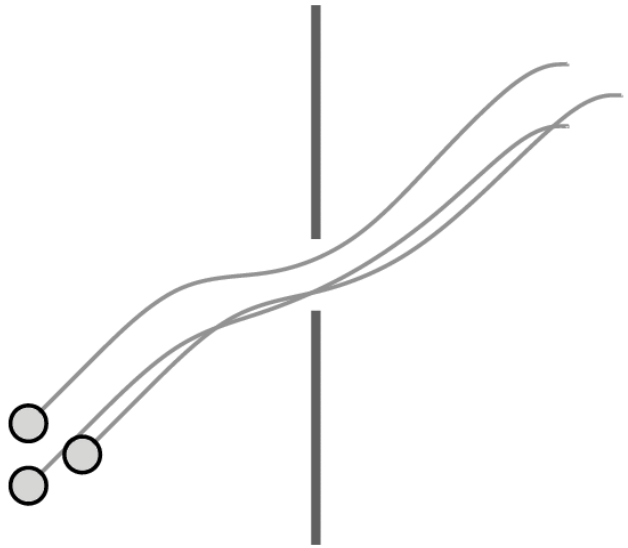
著者：R Van Parys, G Pipeleers

学会：11th Asian Control Conference (ASCC), 2017

目的：動的障害物や狭い通路がある環境下での**隊列走行**

手法：車両ペアごとに全予測ステップの**分離超平面を共有**し、
個別のMPCではそれらを**固定**して分離領域内で行動
超平面の更新方法を2つ（**Method A, B**）提案

結果：2次元・全方向移動車両のシミュレーションで
Method A・Bは**同程度の収束**
通信と計算の少ないMethod Aを実用上推奨



論文1：背景

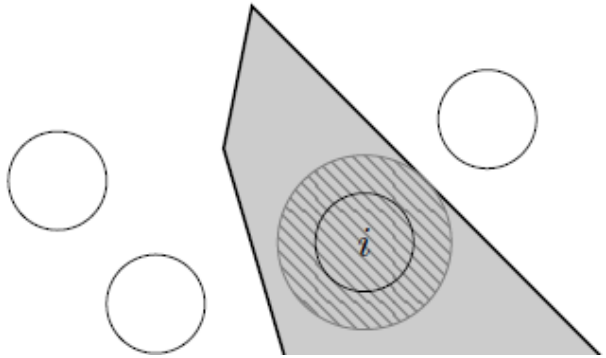
Distributed model predictive control (DMPC) の機体間衝突回避

方式	仕組み	課題
Sequential (逐次型)	機体が**順番に**行動計画 先に計画した機体を回避	・順番によって**優先順位**が発生する ・1制御周期内で各機体が順番に行動計画するので、MPCの**更新が遅い**
Iterative (反復型)	各周期で分散最適化を **収束するまで実行**	・反復ごとに最適化と通信が必要で **更新が遅い**
Synchronous (同期型)	全機体が**同時に1度だけ並列**で計画	・相手の**新しい計画が分からない** ・最も近い機体との距離で行動範囲を制限する場合、**保守的**

本論文は同期型の保守性を分離超平面によって緩和する

速度障害物系の手法は予測ステップ内で他機体の入力が一定

とするため、軌道の最適性が低い



論文1：問題設定

N台の速度入力型･全方向移動車両の点対点移動

目的関数

目標誤差

$$J_i(x_i) = \int_0^T \left\| x_i(t) - x_{T,i} \right\|_1 dt$$

局所的制約

入力の制約

$$\dot{x}_{\min,i} \leq \dot{x}_i \leq \dot{x}_{\max,i}$$

障害物との衝突回避制約

車両と障害物の間の

分離超平面の存在条件

$$-a(t)^{\mathrm{T}} x_i(t) + b(t) \geq r_i,$$

$$a(t)^{\mathrm{T}} x_{\mathrm{o}}(t) - b(t) \geq r_{\mathrm{o}},$$

$$a(t)^{\mathrm{T}} a(t) \leq 1, \ \forall t \in [0, T]$$

大域的制約

隊列制約

$$g_{i,j}(x_i(t), x_j(t)) = x_i(t) - x_j(t) - \Delta x_{i,j} = 0, \forall t \in [0, T]$$

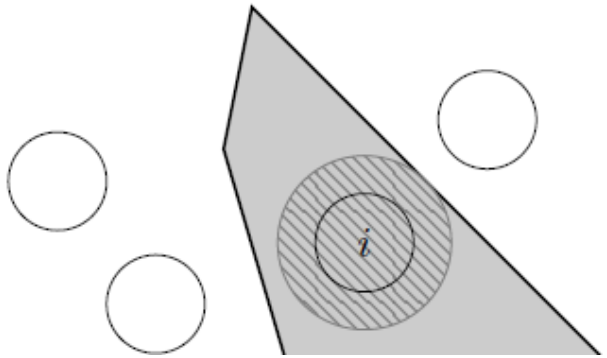
車両間の制約

対障害物と同様

全予測ステップの全車両ペアに対して適用

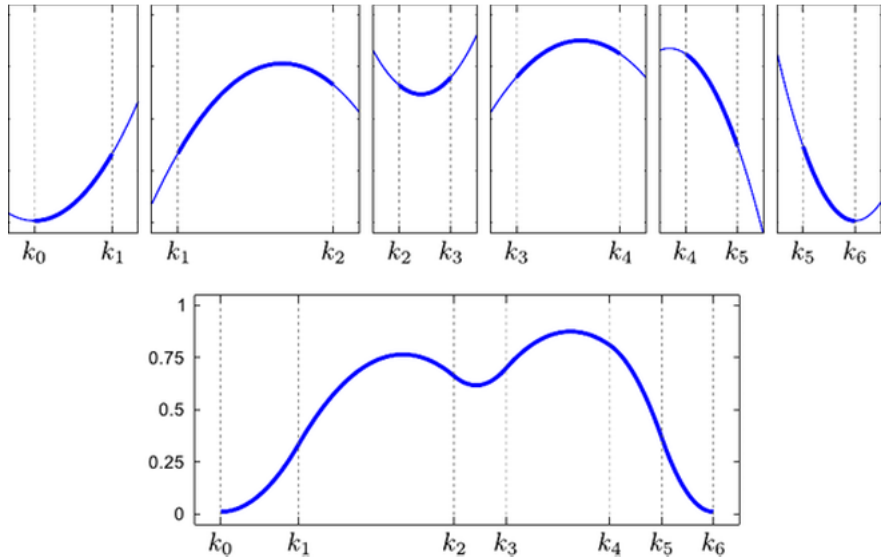
車両iは半径r_iの円で近似

分離超平面$\{\mathrm{x} \in \mathbb{R}^2 | a^{\mathrm{T}} x = b\}$は時間とともに変化してよい



論文1：軌道生成方法

B-スプラインで無限次元問題を有限次元化



B-スプライン

$\hat{x}_i(\mathrm{t}) = \sum_{l=1}^{n} \mathrm{x}_{i,l}\, b_l(t)$

$b = [b_1, \ldots, b_n]^{\mathrm{T}}$： B-スプライン基底

$\mathrm{x}_i = \left[\mathrm{x}_{i,1}, \ldots, \mathrm{x}_{i,n}\right]^{\mathrm{T}}$：係数（最適化変数）

凸包性

基底は正で和が1のとき、スプラインは係数の**凸包**に含まれる

係数に制約を課せば**全時刻で**制約が成り立つ

$\mathrm{x}_i \leq 0 \Longrightarrow \hat{x}_i(\mathrm{t}) \leq 0, \forall t \in [0, T]$

全てスプラインで表す

軌道x_iと超平面の係数$a_{i,\mathrm{j}}, b_{i,\mathrm{j}}$を全てスプライン化

微分・積もスプラインなので、多項式制約は係数の非線形計画になる

論文1：隊列制約の分割手法

ADMM（Alternating Direction Method of Multipliers；交互方向乗数法）で隊列制約を分割し、1周期に1反復だけ回す

x更新
局所制約を満たす自機の軌道計画

通信
xを隊列の近傍と交換

z更新
自機と近傍の「推定軌道」に隊列制約を課す

λ更新
x = zの双対変数λを更新隊列に送る

ポイント

- 車両を動かすのは**x**
- x=zは双対化されているので、隊列制約は実質**ソフト制約**になる
  （反復回数が1回のみであるため）
- 目的地に向かいながらADMMが収束する

本論文の動機

- 動的障害物や狭い通路に対しては**隊列を崩せる**
- その分崩れた時の**機体間衝突回避**が必要になる

論文1：衝突回避制約の分割手法

衝突回避は主空間で分解する

双対分解（ADMM型）

超平面の局所コピーを持ち一致制約を双対化

1周期1反復では一致せず**衝突回避を保証できない**

主空間での分解

周期kの間は超平面を**固定**

各機は自分側の**半空間だけ**で軌道を最適化

半空間の移動制約

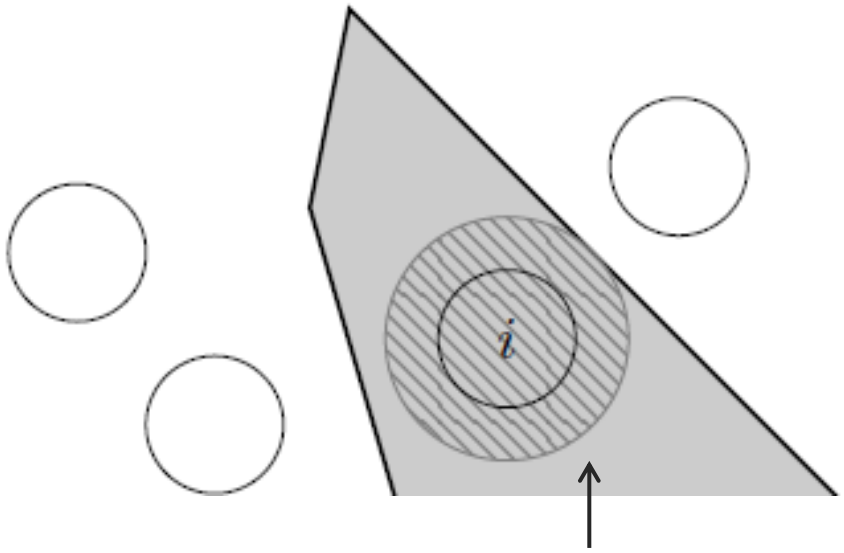
$$-a_{i,j}^{k}(t)^{\mathrm{T}}x_i(t)+b_{i,j}^{k}(t)\geq r_i s_{i,j}(t),$$

$$a_{i,j}^{k}(t)^{\mathrm{T}}a_{i,j}^{k}(t)\leq s_{i,j}(t)^2,$$

$$s_{i,j}(t)\geq 0,\ \forall j\neq i,\ \forall t\in[0,T]$$

超平面の更新に求められる条件

1. **再帰的実行可能性**（次周期にも実行可能解がある）
2. **実行可能領域**をできるだけ広く

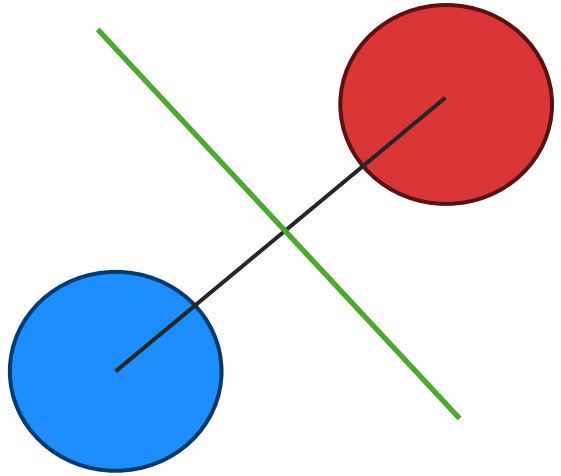


論文1：分離超平面の求め方

Method A: **前回軌道の中点で垂直に2等分**

$$a_{i,j}^{k+1}(\cdot) = x_j^k(\cdot) - x_i^k(\cdot)$$

$$b_{i,j}^{k+1}(\cdot) = \frac{1}{2}\left(x_j^k(\cdot) - x_i^k(\cdot)\right)^{\mathrm{T}}\left(x_j^k(\cdot) + x_i^k(\cdot)\right)$$



空間を等分

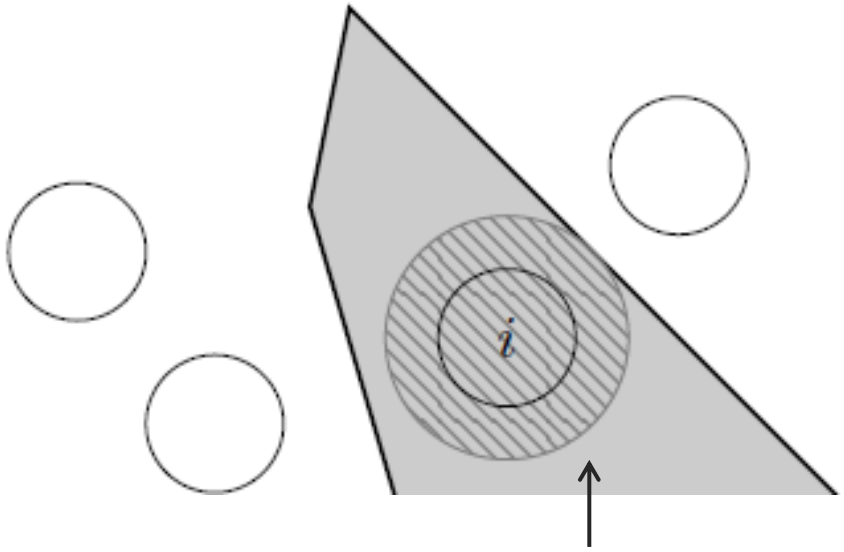
各時刻で2機を結ぶ線分の中点を通り、それに垂直な超平面

再帰的実行可能性

前周期の軌道は次周期でも必ず実行可能な解になる

追加通信なし

Step7で共有済みの近傍の軌道だけで計算できる

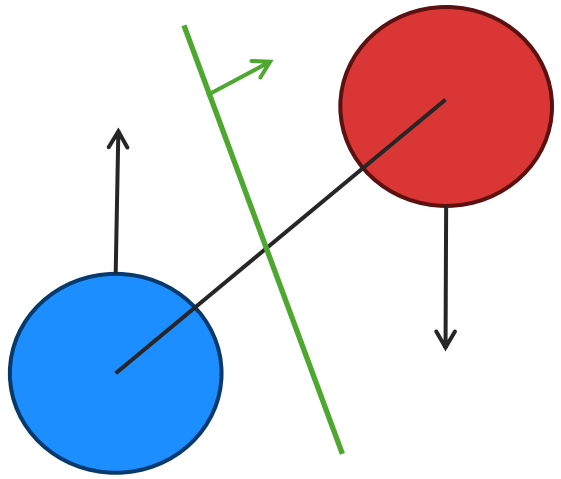


論文1：分離超平面の求め方

Method B:**目的関数の感度で分離超平面を更新**

$$\mathrm{a}_{i,j}^{k+1} = \mathrm{a}_{i,j}^{k} - \alpha_{i,j}^{k}\left(\frac{d\mathcal{L}_{\rho,i}^{k*}}{d\mathrm{a}_{i,j}^{k}} - \frac{d\mathcal{L}_{\rho,j}^{k*}}{d\mathrm{a}_{j,i}^{k}}\right)$$

$$\mathrm{x}_i^k = \underset{\mathrm{x}_i \in \mathrm{X}_i(\hat{\mathrm{x}}_i^k)}{\operatorname{argmin}} \mathcal{L}_{\rho,i}^{k}(\mathrm{x}_i, \mathrm{z}_i^k, \lambda_i^k, \mathrm{x}_i, \mathrm{z}_{j,i}^k, \lambda_{j,i}^k) \quad (7)$$



感度

(7)式の最適目的関数を最適双対変数で微分

両機が「分離超平面をどう動かしたいか」を等しく反映

bの決め方

感度は使わず2機の中点を通るように結ぶ

（感度で更新すると結果が悪かった）

ステップ幅α

前回軌道を分離できる範囲でライン探索

最適的実行可能性をもつ

コスト

感度情報の追加通信と**ライン探索の計算**が必要

論文1：既存手法との比較

再最適化できる範囲が広い

提案手法（グレー領域）

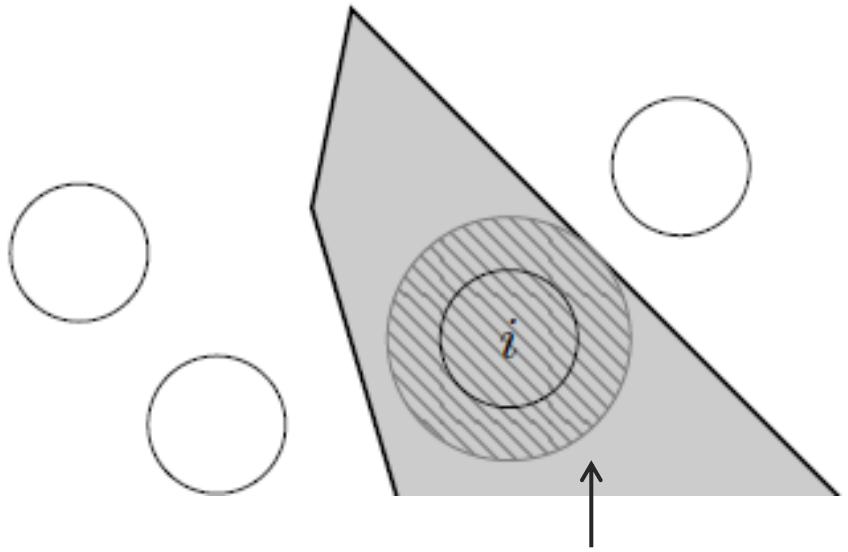
各時刻で近傍の各機体との直線で囲まれた多角形の中なら、

機体iは自由に動ける

既存手法[10,11]（斜線領域）

最も近い機体までの距離で前回軌道からの逸脱を制限

小さな円の中でしか動けない

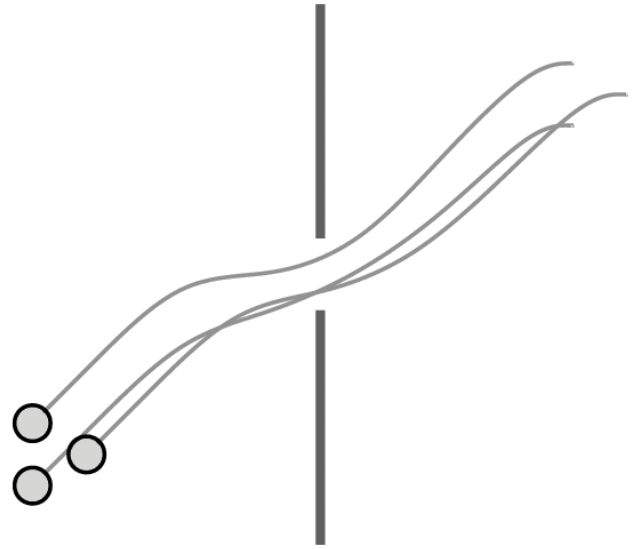


論文1：シミュレーション

狭い通路の通過

制御ホライズンと初期条件を固定し、Method A, Bを比較

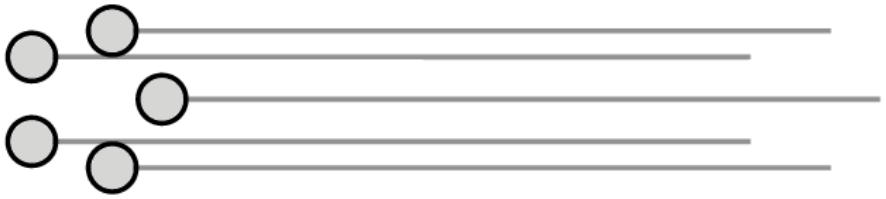
総目的関数J^kと隊列誤差ϵ_{F}^kはほぼ同じ

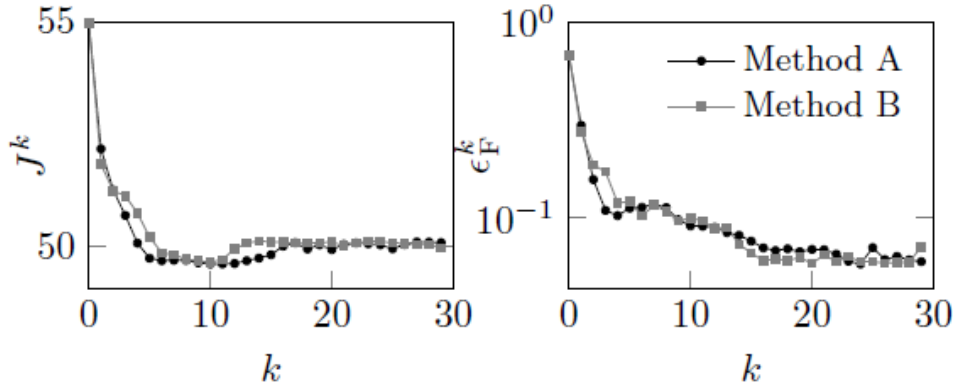


動的障害物の回避

5台の隊列が2つの動的障害物を回避

隊列は崩れたが、衝突は回避できた





性能で優劣がつかない→追加通信とライン探索が不要なMethod Aが実用的

論文1：実機実験



論文1：まとめ

狭所空間や動的環境でリアルタイム隊列走行が可能な分散型MPC (DMPC)を構築

物体間の分離境界線の求め方

- Method A: 前回軌道の中点で垂直に二等分
- Method B: 目的関数の感度で分離超平面を更新
- 従来手法より**移動可能な領域を拡張**

シミュレーション・実機実験

- Method AとBで性能はほとんど変わらないため、通信・計算コストが軽いMethod Aが実用的
- 計算コストが軽いため、実機でもリアルタイム制御可能

自身の研究への応用可能性

- **計算負荷低減**のために、車両間の分離境界線を**SVMのみ**で求めるのではなく、
  本論文のように2つの車両の垂直二等分線を**幾何的に求めて初期値**とする
- **車両間協調**のために**目的関数の感度**を用いてみる

論文2: 概要

Online Trajectory Generation

with Distributed Model Predictive Control

for Multi-Robot Motion Planning

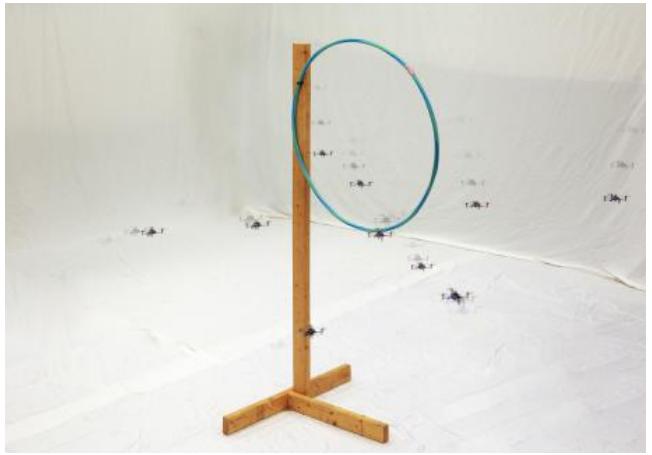
著者：CE Luis, M Vukosavljev, AP Schoellig

学会：IEEE Robotics and Automation Letters, 2020

目的：多数のロボットが**密集**する環境で、
衝突しない軌道を**リアルタイム**に生成する

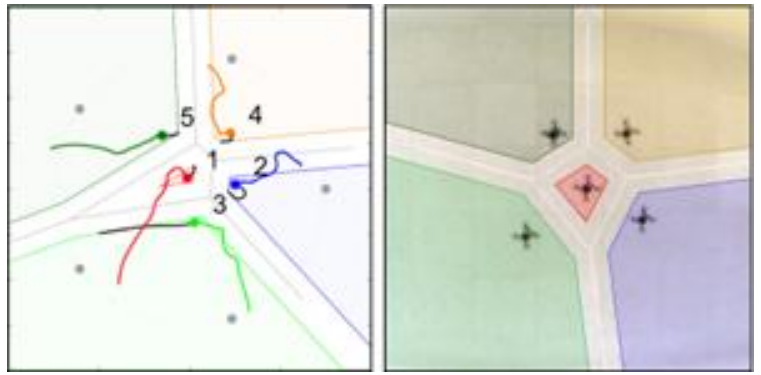
手法：**予測軌道を共有するDMPC**を標準的な**QP**で定式化
衝突が予測された時刻だけを制約する**On-demand回避**と
イベント駆動の再計画を導入

結果：BVC（従来手法）と比較して、移動時間を平均約50%短縮
18 m^3に**30機**で**成功率90%超**
実機20機で検証



$$\begin{aligned} \text{minimize} \quad & \tfrac{1}{2}\mathbf{x}^{\mathrm{T}} Q \mathbf{x} + \mathbf{c}^{\mathrm{T}} \mathbf{x} \\ \text{subject to} \quad & A\mathbf{x} \leq \mathbf{b}, \end{aligned}$$

QP: Quadratic Program（二次計画）

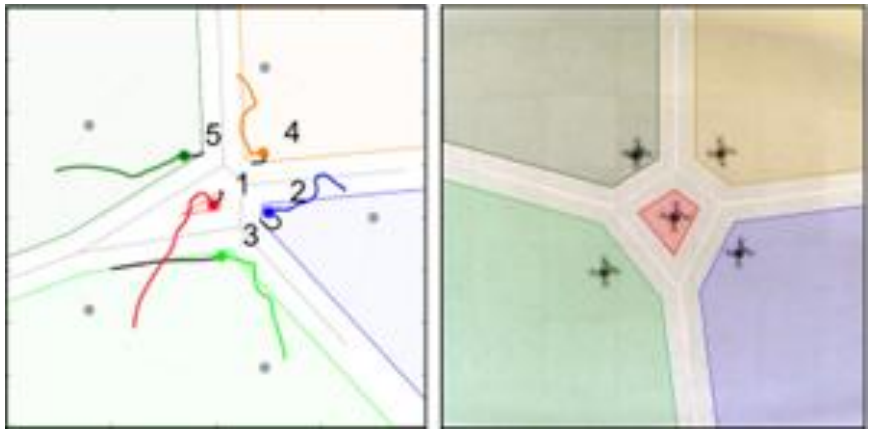


BVC: Buffered Voronoi Cells
（バフ付きボロヌイセル）

論文2：背景

多機軌道生成の既存手法と課題

手法	課題
SCP オフラインDMPC	点対点の計画はできるが、 事前計画のため外乱に適応できない
マルチエージェント RRTなど	オフライン計画が中心 （GPUで高速化する例もある）
ORCA	リアルタイムだが、実証は主に平面環境
BVC	現在位置で空間を分割するため保守的 密集時にデッドロックしやすい
分散チューブMPC	証明はあるが、現状のハードウェアでは リアルタイム実装が難しい



BVCのイメージ

機体間の中点で垂直に二等分された領域（ボロヌイセル）で行動計画を行う

本論文は純粋に最適化ベース（QP）で、

BVCより保守的でない手法を目指す

論文2：問題設定

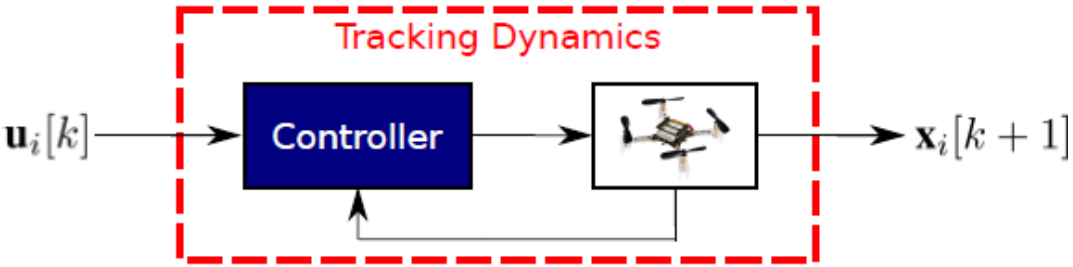
線形な追従ダイナミクスをもつN機の点対点移動

$$\mathbf{x}_i[k+1] = \mathbf{A}_i\mathbf{x}_i[k] + \mathbf{B}_i\mathbf{u}_i[k],$$

$\mathbf{x}_i[k] = (\mathbf{p}_i[k], \mathbf{v}_i[k]) \in \mathbb{R}^6$, 位置と速度

$\mathbf{u}_i[k] \in \mathbb{R}^3$, 位置制御器への位置指令

$\mathbf{A}_i \in \mathbb{R}^{6\times6}$, $\mathbf{B}_i \in \mathbb{R}^{6\times3}$



要件

衝突しない

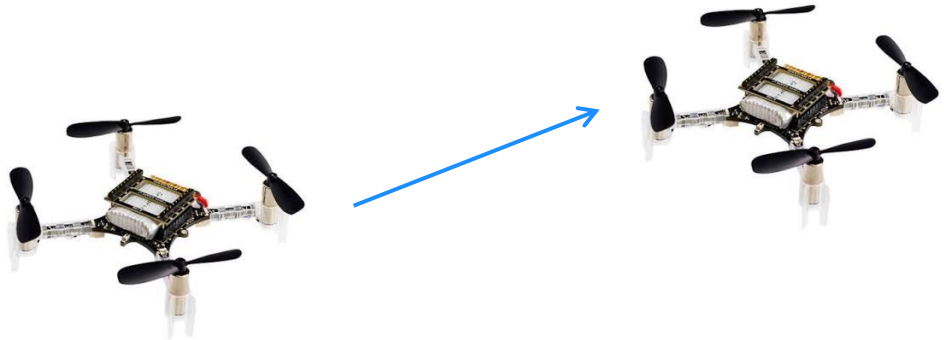
機体同士および静的障害物$\mathcal{E} \in \mathcal{W}$と衝突しない

領域内

常に作業空間$\mathcal{W} \in \mathbb{R}^3$の中にとどまる

到達

ある時刻T_f以降目標位置$\mathbf{p}_{d,i} \in \mathcal{W}$の十分近くにいる



論文2：軌道生成と予測方法

入力をBézier curve（ベジェ曲線）で表し、予測を線形に

Bernstein polynomial（ベルンシュタイン多項分布）

$$S_{m,p}(t) = \binom{p}{m}(1-t/T)^{p-m}(t/T)^{m},\ \forall t \in (0,T),\ m = 0,1,\ldots,p$$

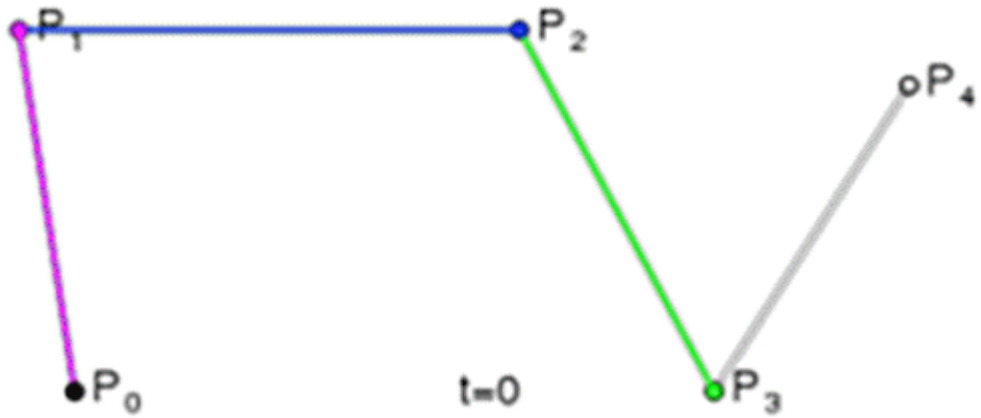
3次元空間における次数pのベジェ曲線

$$\mathbf{S}(\mathrm{t}) = \sum_{i=0}^{p} \mathbf{P}_m S_{m,p}(t),\ \mathbf{P}_m \in \mathbb{R}^3$$

制御点の集合を$\mathcal{P} = \{\mathbf{P}_0, \mathbf{P}_1, \ldots, \mathbf{P}_p\}$とする（$p+1$個）

軌道生成にはベジェ曲線をl本連結

入力は軌道上の制御点$\boldsymbol{\mathcal{U}}_i \in \mathbb{R}^{3l(p+1)}$で最適化変数とする



予測モデル

$$\mathbf{X}_i = \mathbf{A}_{0,i}\bar{\mathbf{x}}_i[k_t] + \Lambda_i \mathbf{U}_i$$

初期制御点の固定と曲線のつなぎ目での微分一致のために

等式制約$\mathbf{A}_{\mathrm{eq}}\boldsymbol{\mathcal{U}}_i = \mathbf{b}_{\mathrm{eq}}$を課す

論文2：衝突回避

BVCとOn-demandはどちらも超平面で移動可能空間を制限する

衝突回避の目標：$\left\|\boldsymbol{\Theta}^{-1}(\mathbf{p}_i[k_t]-\mathbf{p}_j[k_t])\right\|_2 \geq r_{\min},\ \forall j \neq i$

$\boldsymbol{\Theta}^{-1}$：楕円体の安全境界を作るスケーリング行列

BVC (Buffered Voronoi Cell)

現在の**計測位置**から各機のVoronoiセル$\mathcal{V}_i$を作る

最初のベジェ曲線区間の制御点を**全て**$\mathcal{V}_i$に入れる

（近傍ごとに$p+1$本の制約付き）

必要な情報：全機の現在位置のみ（**通信が不要**）

On-demand

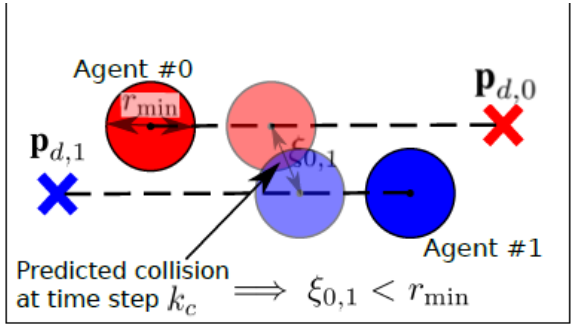
前周期の**予測軌道**で最初に衝突する時刻$k_{c,i}$を検出

その**1サンプルだけ**に線形化した制約を課す

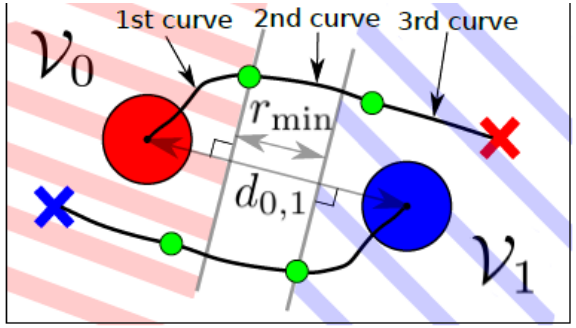
（近傍ごとに1本、ソフト制約用スラック変数ε_{ij}付き）

必要な情報：近傍の予測軌道（**通信が必要**）

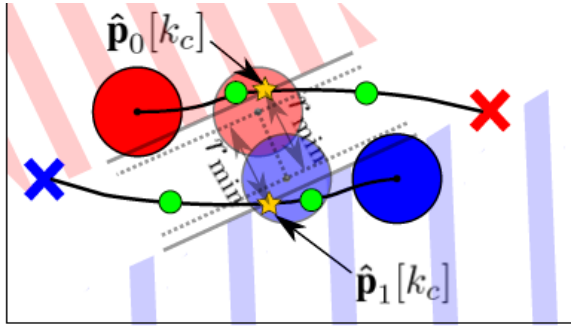
On-demand衝突回避：$\left\|\boldsymbol{\Theta}^{-1}(\hat{\mathbf{p}}_i[k_{c,i}-1|k_t]-\hat{\mathbf{p}}_j[k_{c,i}|k_t-1])\right\|_2 \geq r_{\min}+\varepsilon_{ij},\ \varepsilon_{ij}<0,\ \forall j \in \Omega_i$



(a) Colliding scenario



(b) BVC collision avoidance



(c) On-demand collision avoidance

論文2：最適制御問題

各機が解くのは標準的なQP

目的関数

目標誤差

$$\mathcal{J}_{i,\mathrm{error}} = \sum_{k=K-\kappa}^{K} q_k \left\| \hat{\mathbf{p}}_i[k|k_t] - \mathbf{p}_{d,i} \right\|_2^2$$

エネルギー

$$\mathcal{J}_{i,\mathrm{energy}} = \sum_{c=0}^{r} \alpha_c \int_0^{t_h} \left\| \frac{d^c}{dt^c} \hat{\mathbf{u}}_i(t) \right\|_2^2 dt$$

衝突回避制約の違反ペナルティ

$$\mathcal{J}_{i,\mathrm{violation}} = \zeta \left\| \varepsilon_{ij} \right\|_2^2 + \xi \varepsilon_{ij}$$

制約

ベジェ曲線の接続

$$\mathbf{A}_{\mathrm{eq}} \boldsymbol{\mathcal{U}}_i = \mathbf{b}_{\mathrm{eq}}$$

物理的制約

$$\gamma_{\min}^{(2)} \leq \frac{d^2}{dt^2} \mathbf{u}_i\,(t) \leq \gamma_{\max}^{(2)}$$

衝突回避制約

$$\left\| \boldsymbol{\Theta}^{-1} (\hat{\mathbf{p}}_i[k_{c,i} - 1|k_t] - \hat{\mathbf{p}}_j[k_{c,i}|k_t - 1]) \right\|_2 \geq r_{\min} + \varepsilon_{ij},$$

$$\varepsilon_{ij} \leq 0,\ \forall j \in \Omega_i$$

決定変数は制御点$\boldsymbol{\mathcal{U}}_i$とスラック$\varepsilon_{ij}$
衝突回避制約が**ソフト制約**なので**常に解が得られる**が、**衝突回避は保証されない**

論文2：シミュレーション

4手法をランダムな移動タスクで比較

1. **BVC**
   離散計画器なし
2. **Soft BVC**
   BVCをソフト制約化
3. **On-demand (state)**
   状態空間
4. **On-demand (input)**
   入力空間

環境

障害物がない18 m^3の作業空間、ドローン10~60機、

ランダムに初期地点と最終地点を変えて50ケース



成功の定義

20 s以内に全機が衝突なく目標から10 cm以内に到達

機体

Crazyflie 2.0： $r_{\min} = 0.3\ \mathrm{m}$（計画用）、$r_{\mathrm{col}} = 0.2\ \mathrm{m}$（衝突判定）、計測ノイズあり

MPC

制御周期：$h = 0.2\ \mathrm{s}\ (5\ \mathrm{Hz})$、 ベジェ曲線：$p = 5, l = 3, t_h = 3\ \mathrm{s}$

論文2：シミュレーション結果

入力空間のOn-demandが密集環境で最も高性能

成功率

入力空間のOn-demandは30機まで**90%超**

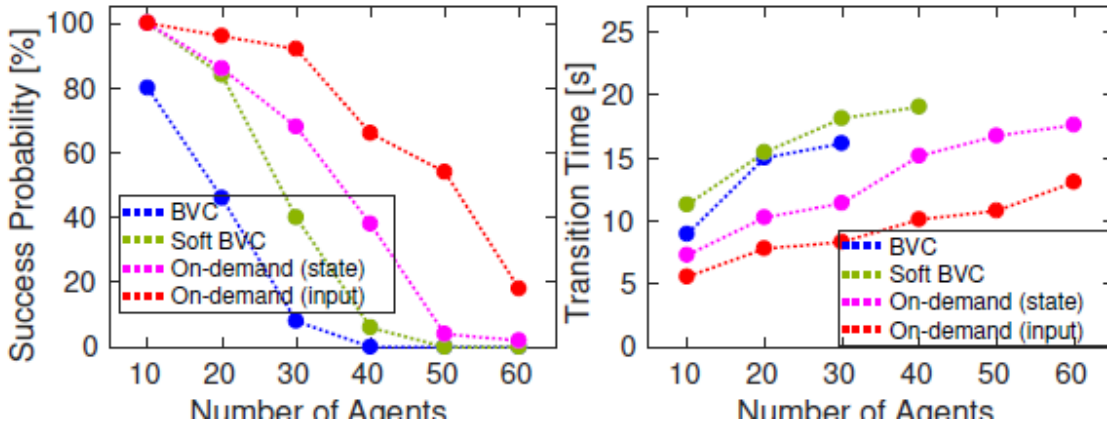
30機を超えると全手法で大幅に低下

移動時間

BVCの手法に比べて**On-demand**は**約50%短縮**

移動距離

Soft BVCが約3%短いが、全手法でほぼおなじ



BVCが弱い理由

密度が上がると保守的すぎて移動ができなくなる

ソフト制約にしても限度がある

入力空間が良い理由

位置は入力より遅れているので、先に回避できる

モデル誤差にも強い

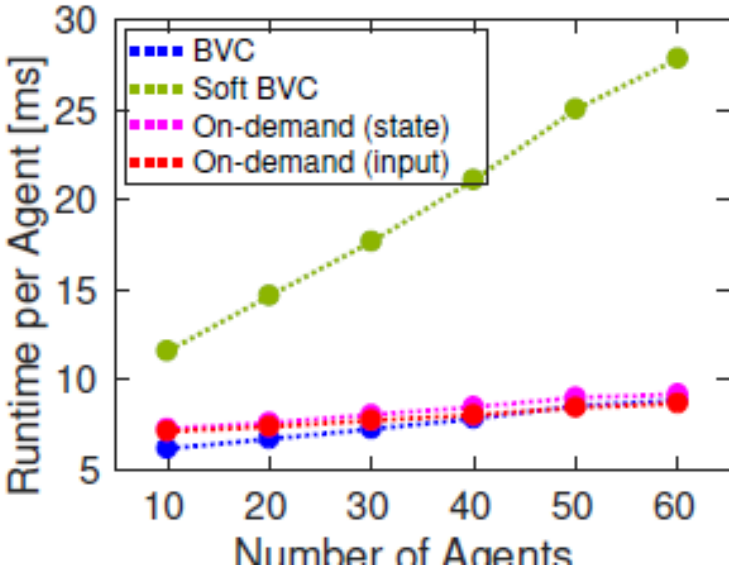
位置と入力が別物の系では衝突回避が保証できない

論文2：シミュレーション結果

一機当たりのQP計算時間はBVCとほぼ同等

手法	衝突回避制約の数	追加変数
BVC	$\mathcal{O}(N_{i,k_t})$	なし
Soft BVC	$\mathcal{O}(N_{i,k_t})$	$(p+1)N_{i,k_t}$個のスラック
On-demand	$\mathcal{O}(N_{i,k_t})$	N_{i,k_t}個のスラック

N_{i,k_t}: 衝突回避で考慮する近傍の機体数



Soft BVCが遅い理由

最も多くのスラック変数を必要とする

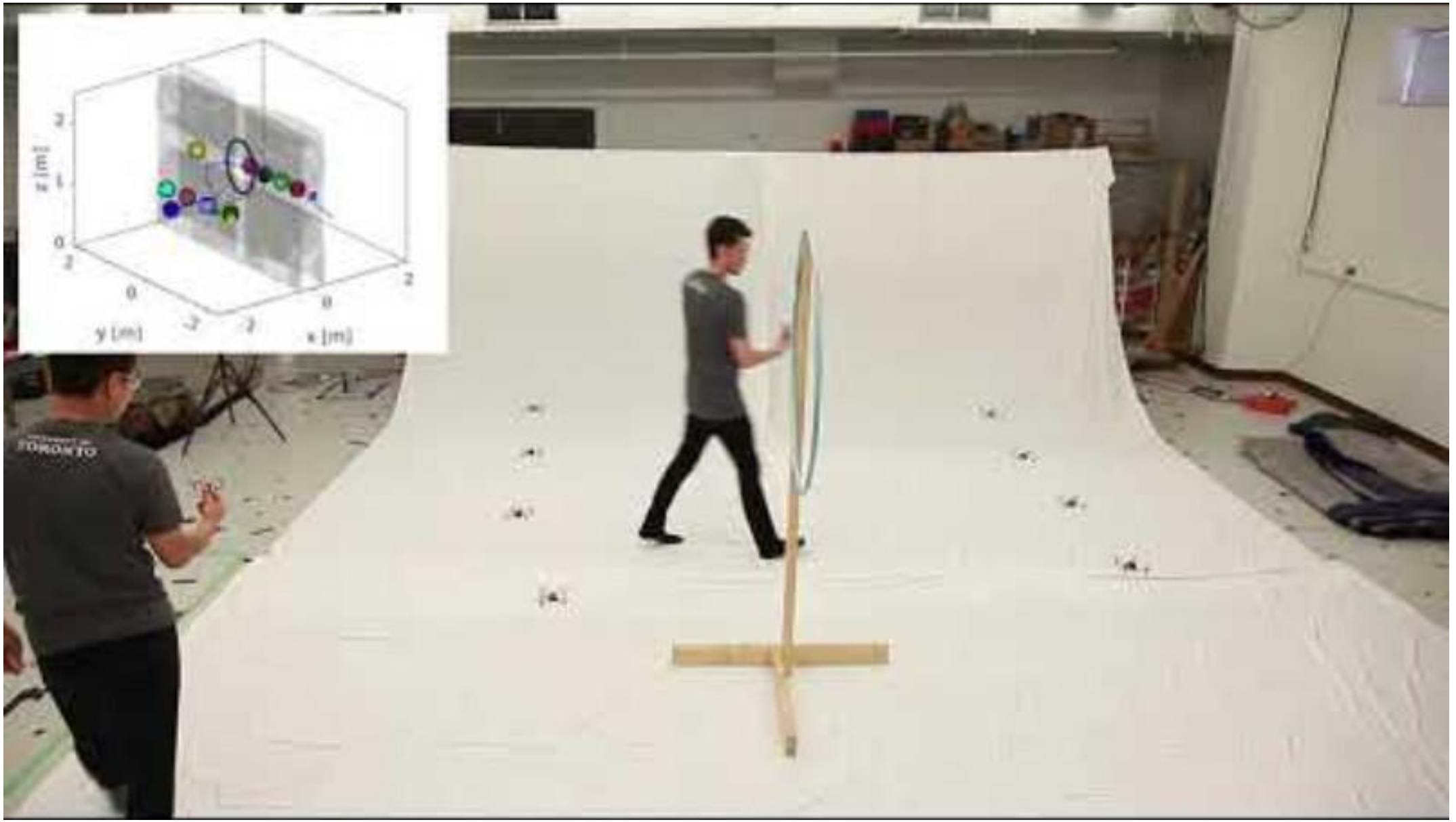
=最も問題が大きい

BVCが速い理由

追加変数が不要

On-demandは追加のスラック変数が少ない

論文2：実機実験



論文2：実機実験結果（障害物なし）

20機まで計算時間はほぼ線形に増加

# Agents	2	4	6	8	10	12	14	16	18	20
Avg. solve time of Alg. 1 [ms]	3.3	6.2	9.9	13.8	16.9	17.6	20.3	20.5	23.4	28.3
Std. solve time of Alg. 1 [ms]	0.4	0.9	1.7	3.3	5.1	7.6	8.0	8.9	10.2	11.4
Min. distance [cm]	36.2	32.2	31.1	30.0	29.2	28.6	29.1	26.0	26.1	25.3

実装

C++・ROS・qpOASESを使用

1台のPCが全機分（Alg. 1全体）を計算し、20 Hzで無線送信

$3 \times 3 \times 2\ \mathrm{m}^3$の作業空間、 $r_{\min} = 0.35\ \mathrm{m}$

最小距離

$r_{\min} = 0.35\ \mathrm{m}$を**下回る**ことがある（**ソフト制約**のため）が、**衝突はしなかった**

計算時間

ほぼ線形に増加（全機分を**1台の計算機で計算**しているため）

論文2：実機実験結果（障害物あり）

10機がフラフープを通って位置交換

設定

直径85 cmのフープ以外を壁で遮る

障害物は交差する4つの楕円体で、通れる隙間は$30 \times 30\ \mathrm{cm}^2$

障害物の扱い

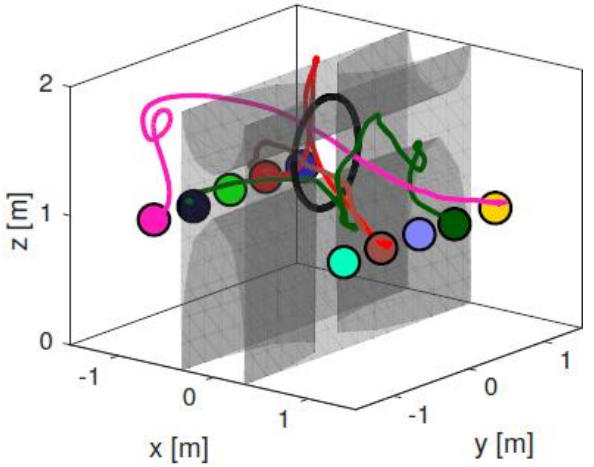
静的障害物を「近傍」として追加するだけ

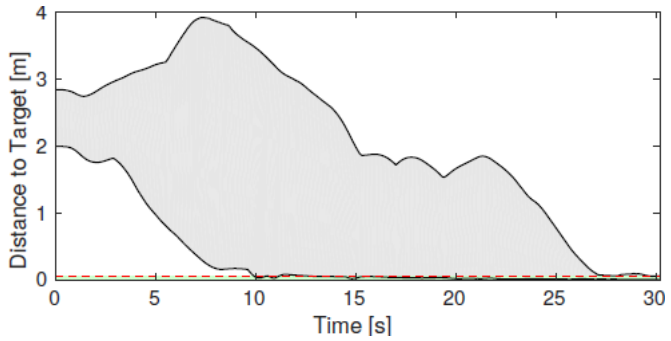
計算量は障害物の数に対して線形に増加

結果

直接通る機体と迂回して譲る機体が現れた

18試行で移動完了時間は20.1~48.4 s





チューニングへの頑健さ

入力空間では$\xi \in [-1.5, -0.5] \times 10^3$で幅広く成功

$\xi < -2 \times 10^3$では障害物付近で振動しデッドロックすることも

課題

状態空間ではデッドロックが多発

非凸な障害物では局所解に陥るため

離散計画による経由点が必要になりうる

論文2：まとめ

混雑環境や狭所環境でリアルタイム移動が可能な分散型MPC（DMPC）を構築

衝突回避手法

- BVC：現在の計測位置からボロヌイセルを作成し、移動可能な空間とする
- Soft BVC：BVCの制約をスラック変数を使ってソフト緩和
- On-demand：前周期の予測軌道で最初に衝突が発生した時刻のみに、衝突回避用の超平面を与える

シミュレーション・実機実験

- 入力空間でのOn-demandが成功率が最も高く、移動時間も最短になった
- Soft BVCはスラック変数が多いため、計算時間が長くなった

自身の研究への応用可能性

- **計算時間の短縮**のために、**決定変数や制約の数**を減らせるアルゴリズムを考える
  例えば、本論文のように**衝突しそうな車両だけ**衝突回避を考慮するなど
- **保守的**すぎない解を出すために衝突回避制約の**ソフト緩和**をしてみる